

Chapter 13: `lower_bound()`

This is one of the most important STL functions for interviews.

Many developers use:

```
binary_search()
```

But experienced interview candidates often use:

```
lower_bound()
```

because it's more powerful.

What Does `lower_bound()` Do?

Definition:

Returns an iterator pointing to the **first element** $\geq x$

Remember this:

```
lower_bound(x)
=
first element  $\geq x$ 
```

Example 1

```
1 3 5 7 9
```

Find:

```
lower_bound(5)
```

Answer:

```
5
```

Because:

```
5 >= 5
```

and it's the first such element.

Example 2

```
1 3 5 7 9
```

Find:

```
lower_bound(4)
```

Answer:

```
5
```

Because:

```
5 >= 4
```

and it's the first such element.

Example 3

```
1 3 5 7 9
```

Find:

```
lower_bound(8)
```

Answer:

```
9
```

Example 4

```
1 3 5 7 9
```

Find:

```
lower_bound(20)
```

No element satisfies:

```
>= 20
```

Returns:

```
end()
```

Header

```
#include <algorithm>
```

Syntax

```
lower_bound(begin,end,x)
```

Returns:

```
iterator
```

Not a value.

Basic Example

```
vector<int> v={1,3,5,7,9};

auto it = lower_bound(
    v.begin(),
    v.end(),
    4
);

cout << *it;
```

Output:

```
5
```

Position (Index)

Very important.

```
vector<int> v={1,3,5,7,9};

auto it=lower_bound(
    v.begin(),
    v.end(),
    4
);

cout << it-v.begin();
```

Output:

2

Because:

Index:

```
0 1 2 3 4
1 3 5 7 9
      ^
```

Finding Insert Position

Interview favorite.

Input:

1 3 5 7 9

Insert:

4

Where?

lower_bound(4)

returns index:

2

So insert at position 2.

Result:

```
1 3 4 5 7 9
```

Complexity

```
O(log n)
```

Duplicate Elements

This is where lower_bound becomes very useful.

Input:

```
1 2 2 2 2 5 7
```

Find:

```
lower_bound(2)
```

Answer:

```
first 2
```

Index:

```
1
```

Visualization:

```
1 2 2 2 2 5 7
  ^
```

Why Important?

Because it finds the:

```
FIRST occurrence
```

among duplicates.

Compare with Binary Search

Array:

```
1 2 2 2 2 5
```

Binary Search:

```
binary_search(...)
```

Returns:

```
true
```

or

```
false
```

Only existence.

Lower Bound:

```
lower_bound(...)
```

Returns:

```
iterator
```

Much more information.

Using With Set

Set has member function.

```
set<int> s={10,20,30,40};  
  
auto it=s.lower_bound(25);  
  
cout<<*it;
```

Output:

```
30
```

Complexity:

```
O(log n)
```

Using With Map

```
map<int,string> mp;  
  
mp[10]="A";  
mp[20]="B";  
mp[30]="C";
```

```
auto it=mp.lower_bound(25);
```

Points to:

30 -> C

Access:

```
cout<<it->first;  
cout<<it->second;
```

Common Interview Trick

Count elements smaller than x.

Example:

1 3 5 7 9

Find count of elements:

< 6

Code:

```
auto it=lower_bound(  
    v.begin(),  
    v.end(),  
    6  
);  
  
cout<<it-v.begin();
```

Output:

3

Because:

1 3 5

are smaller.

Common Interview Trick

Count elements:

```
>= x
```

Example:

```
1 3 5 7 9
```

x:

```
5
```

Code:

```
auto it=lower_bound(
    v.begin(),
    v.end(),
    5
);

cout<<v.end()-it;
```

Output:

```
3
```

Elements:

```
5 7 9
```

Manual Meaning

```
lower_bound(x)
```

means:

```
First Position Where x Can Be Inserted  
Without Breaking Sorting
```

This definition is very useful.

Practice Questions

Easy

Q1

Find lower_bound of:

```
1 3 5 7 9
```

for:

```
4
```

Answer:

```
5
```

Q2

Print index of lower_bound(7).

Q3

Check if lower_bound exists.

Hint:

```
if(it!=v.end())
```

Q4

Insert element using lower_bound position.

Q5

Count elements less than x.

Medium

Q6

Find first occurrence of x.

Input:

```
1 2 2 2 2 5
```

x:

```
2
```

Answer:

```
1
```

Q7

Count elements $\geq x$.

Q8

Search in sorted vector of strings.

Use:

```
lower_bound()
```

Q9

Find nearest element $\geq x$.

Q10

Implement Search Insert Position.

LeetCode #35

Interview Level

Q11

Search Insert Position

LeetCode #35

Q12

Find First and Last Position

LeetCode #34

(lower_bound + upper_bound)

Q13

Count Occurrences in Sorted Array

Q14

Aggressive Cows

(Binary Search + lower_bound idea)

Common Mistakes

Mistake 1

Using unsorted vector.

```
5 1 7 3
```

✗ Wrong

Must be sorted.

Mistake 2

Dereferencing end()

```
*it
```

when:

```
it==v.end()
```

✗ Runtime Error.

Mistake 3

Thinking it returns value.

Wrong:

```
int x=lower_bound(...);
```

✗

Returns iterator.

Revision Sheet

Header

```
#include <algorithm>
```

Syntax

```
lower_bound(  
    begin,  
    end,  
    x  
);
```

Meaning

First Element $\geq x$

Return

iterator

Index

```
it-v.begin()
```

(vector only)

Complexity

$O(\log n)$

Works On

```
vector  
array  
set  
map  
multiset
```

(sorted containers)

The Formula You Must Memorize

```
lower_bound(x)
=
first element >= x
```
